

Guia de Estudo: TypeScript, Lógica e Modelagem para Entrevistas QA

Para: Higor Mesquita
Objetivo: Desenvolver capacidade de modelar sistemas em TypeScript, resolver problemas de lógica e performar melhor em entrevistas técnicas (cliente/VP).
Cursos base: *Understanding TypeScript* (Maximilian Schwarzmüller) + *Playwright JS/TS Automation* (Rahul Shetty Academy)
Versão: 1.0 — Agosto/2026

Sumário

- 1. [Diagnóstico: o que faltou na entrevista do elevador](#)
- 2. [Mapa de competências](#)
- 3. [Como usar seus cursos Udemy](#)
- 4. [Fundamentos TypeScript para modelagem](#)
- 5. [OOP e estrutura de aplicação](#)
- 6. [Máquina de estados](#)
- 7. [Design Patterns essenciais](#)
- 8. [Algoritmos e estruturas para scheduling](#)
- 9. [Framework de resposta em entrevista](#)
- 10. [Problemas clássicos para praticar](#)
- 11. [Exercício completo: Elevador em TypeScript](#)
- 12. [Plano de estudo 30 dias](#)
- 13. [Checklist pré-entrevista](#)
- 14. [Recursos complementares](#)

1. Diagnóstico {#1-diagnóstico}

O que você disse ao VP

- `type` para andares com limites
- Condicional de direção (subindo → priorizar andares acima)
- Classe `main` com funções
- Validação de peso/capacidade

O que estava correto

- Pensamento em constraints (limites, capacidade)
- Noção de priorização por direção
- Uso de TypeScript

O que faltou (e costuma derrubar candidatos)

Gap	Por que importa
Objetos com papéis claros	VP quer ver arquitetura mental , não só if/else
Máquina de estados	Elevador = estados + transições (porta, movimento)
Fila de requisições	Botão externo vs interno, ordem de paradas
Controller/Scheduler	Quem decide vs quem executa
Regras de porta	Segurança e invariantes do sistema
Visão QA	Edge cases = seu diferencial na vaga

Meta deste guia

Sair de "eu faria um if" para "eu modelaria assim" em 30 dias de estudo focado.

2. Mapa de competências {#2-mapa-de-competências}

NÍVEL 1 – Sintaxe TS	→ types, interfaces, classes, enums
NÍVEL 2 – Modelagem	→ entidades, estados, regras de negócio
NÍVEL 3 – Arquitetura simples	→ Controller, filas, patterns
NÍVEL 4 – Entrevista	→ falar estrutura + testar edge cases

Seu perfil atual: forte em Nível 1 (Playwright/TS no dia a dia), fraco em Nível 2-4 para problemas abertos.

3. Como usar seus cursos Udemy {#3-como-usar-seus-cursos-udemy}

Curso 1: Understanding TypeScript (Maximilian)

Seção do curso	Use para	Exercício extra
Types & Basics	Literal types, unions	Modelar <code>Direction = 'UP' 'DOWN' 'IDLE'</code>
Classes	Encapsulamento, métodos	Classe <code>Elevator</code> com <code>move()</code> , <code>openDoor()</code>
Interfaces	Contratos entre objetos	<code>interface ElevatorRequest</code>
Generics	Coleções tipadas	<code>Queue<ElevatorRequest></code>
Modules	Organizar arquivos	<code>elevator/</code> , <code>controller/</code> , <code>types/</code>
Decorators (se houver)	Pular por ora	—

Regra: ao terminar cada módulo de classes/interfaces, pause e modele um objeto do mundo real (semáforo, elevador, fila).

Curso 2: Playwright JS/TS (Rahul Shetty)

Seção do curso	Use para	Exercício extra
JS/TS fundamentals	Reforçar base	Resolver exercícios de lógica em TS puro
Page Object Model	Estrutura de aplicação	Mesma ideia: separar responsabilidades
Framework from scratch	Organização de projeto	Aplicar em projeto <code>logic-practice/</code>
API testing	Request/Response modeling	Modelar <code>ElevatorRequest</code> como API payload
Cucumber/BDD	Given/When/Then	Escrever cenários do elevador antes de codar

Insight chave: POM (Page Object Model) = mesma skill de entrevista.

- Page ≈ Elevator
- Test ≈ Controller
- Locator ≈ State

4. Fundamentos TypeScript para modelagem {#4-fundamentos-typescript-para-modelagem}

4.1 Literal types para estados

```
type Direction = 'UP' | 'DOWN' | 'IDLE';
type DoorState = 'OPEN' | 'CLOSED' | 'OPENING' | 'CLOSING';
type RequestType = 'HALL_UP' | 'HALL_DOWN' | 'CABIN';
```

Por quê: o compilador impede estados inválidos (`direction = 'SIDEWAYS'`).

4.2 Interface vs Type

```
interface ElevatorRequest {
  floor: number;
  type: RequestType;
  timestamp: number;
}

type FloorNumber = number; // alias semântico
```

Regra prática: `interface` para objetos/contratos; `type` para unions e aliases.

4.3 Classes com responsabilidade única

```
class Door {
  private state: DoorState = 'CLOSED';

  isOpen(): boolean {
    return this.state === 'OPEN';
  }

  open(): void {
    this.state = 'OPEN';
  }

  close(): void {
    this.state = 'CLOSED';
  }
}
```

4.4 Readonly e validação na construção

```
class BuildingConfig {
  readonly minFloor: number;
  readonly maxFloor: number;
  readonly maxCapacity: number;

  constructor(min: number, max: number, capacity: number) {
    if (min >= max) throw new Error('Invalid floor range');
    this.minFloor = min;
    this.maxFloor = max;
    this.maxCapacity = capacity;
  }
}
```

```

    isValidFloor(floor: number): boolean {
      return floor >= this.minFloor && floor <= this.maxFloor;
    }
  }
}

```

4.5 Exercício rápido (15 min)

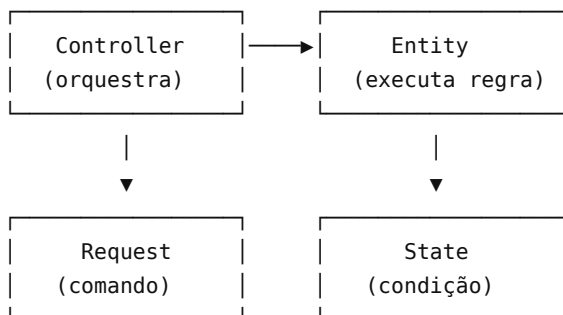
Crie `TrafficLight` com estados `RED` | `YELLOW` | `GREEN` e método `tick()` que transiciona corretamente.

5. OOP e estrutura de aplicação {#5-oop-e-estrutura-de-aplicação}

5.1 Perguntas que você deve fazer ANTES de codar

1. **Quais são as entidades?** (substantivos do problema)
2. **O que cada uma guarda?** (dados)
3. **O que cada uma faz?** (comportamentos)
4. **Quem fala com quem?** (dependências)
5. **Quais regras nunca podem ser quebradas?** (invariantes)

5.2 Template de objetos (use em qualquer problema)



5.3 SOLID — só o que importa para entrevista

Princípio	Aplicação
S — Single Responsibility	Elevator move; Controller agenda
O — Open/Closed	Trocar algoritmo SCAN por nearest via Strategy
D — Dependency Inversion	Controller depende de interface, não implementação

5.4 Estrutura de pastas sugerida

```

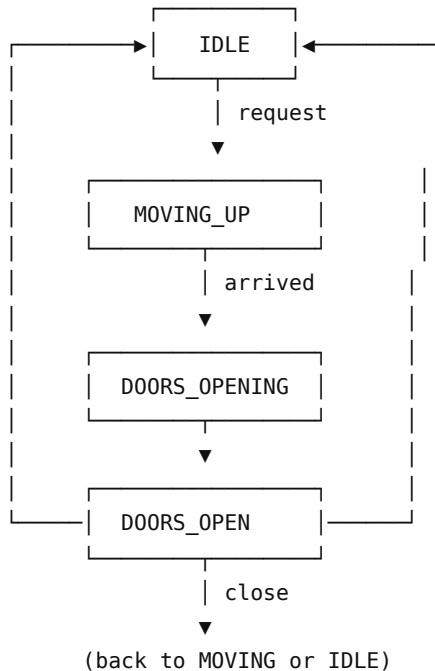
src/
  elevator/
    types.ts      # Direction, DoorState, Request
    door.ts       # Door class
    elevator.ts   # Elevator class
    controller.ts # ElevatorController
    scheduler.ts  # pickNextStop logic
  exercises/
    traffic-light.ts
    vending-machine.ts
    parking-lot.ts

```

```
tests/
  elevator.spec.ts # Playwright ou Vitest/Jest
```

6. Máquina de estados {#6-máquina-de-estados}

6.1 Diagrama do elevador



6.2 Invariantes (regras que NUNCA quebram)

- Não move com porta aberta
- Não move acima do peso máximo
- Só abre porta quando parado no andar correto
- Andar deve estar dentro do range do prédio

6.3 Implementação mínima

```
type ElevatorState = 'IDLE' | 'MOVING' | 'STOPPED';

class Elevator {
  state: ElevatorState = 'IDLE';
  currentFloor = 0;
  direction: Direction = 'IDLE';

  move(): void {
    if (this.state !== 'MOVING') return;
    // lógica de movimento
  }

  stopAt(floor: number): void {
    this.currentFloor = floor;
    this.state = 'STOPPED';
    this.direction = 'IDLE';
  }
}
```

```
}  
}
```

7. Design Patterns essenciais {#7-design-patterns-essenciais}

Prioridade de estudo (nesta ordem)

1. State Pattern

Quando: objeto muda comportamento conforme estado (elevador, porta).

Frase de entrevista: "Eu usaria State para isolar comportamento de cada fase do elevador."

2. Strategy Pattern

Quando: trocar algoritmo de scheduling (SCAN vs nearest).

Frase: "O Controller usa Strategy para escolher próxima parada."

3. Command Pattern

Quando: cada botão pressionado = comando enfileirado.

Frase: "Cada request vira um Command na fila."

4. Facade / Controller

Quando: simplificar interface para o "mundo externo".

Frase: "ElevatorController é a Facade — recebe pedidos e coordena."

5. Observer (opcional)

Quando: displays/painéis precisam atualizar quando andar muda.

Recurso: refactoring.guru/design-patterns (grátis, visual)

8. Algoritmos e estruturas {#8-algoritmos-e-estruturas}

8.1 Estruturas que você precisa dominar

Estrutura	Uso no elevador
Set<number>	Paradas pendentes (sem duplicata)
Queue<Request>	Fila FIFO de pedidos
Array ordenado	Próximas paradas na direção atual
Map	Elevador → fila de paradas

8.2 Algoritmo SCAN (elevador clássico)

1. **IDLE:** vai ao pedido mais próximo
2. **MOVING_UP:** atende todos os andares $\geq$ atual, em ordem crescente
3. **MOVING_DOWN:** atende todos os andares $\leq$ atual, em ordem decrescente
4. Quando fila da direção esvazia → inverte ou fica IDLE

```
function pickNextStop(  
  current: number,  
  direction: Direction,  
  stops: number[]  
) : number | null {
```

```

if (stops.length === 0) return null;

if (direction === 'UP') {
  const above = stops.filter(f => f >= current).sort((a, b) => a - b);
  if (above.length) return above[0];
}
if (direction === 'DOWN') {
  const below = stops.filter(f => f <= current).sort((a, b) => b - a);
  if (below.length) return below[0];
}
// IDLE: nearest
return stops.sort(
  (a, b) => Math.abs(a - current) - Math.abs(b - current)
)[0];
}

```

9. Framework de resposta em entrevista {#9-framework-de-resposta}

Passo a passo (2–3 minutos falando)

1. Clarificar requisitos (30s)

"Antes de modelar: quantos elevadores? Quantos andares? Só capacidade por peso ou também por pessoas? Precisa suportar múltiplos prédios?"

2. Nomear objetos (30s)

"Eu separaria em: Elevator, Door, ElevatorRequest, ElevatorController e BuildingConfig."

3. Explicar responsabilidades (45s)

"Elevator executa movimento e porta. Controller recebe requests e agenda paradas. Request representa botão do hall ou cabine."

4. Regras e estados (45s)

"Estados: IDLE, MOVING_UP, MOVING_DOWN, DOORS_OPEN. Invariantes: não move com porta aberta, respeita capacidade."

5. Algoritmo (30s)

"Scheduling estilo SCAN: na subida, atendo andares acima em ordem."

6. QA angle (30s) ← SEU DIFERENCIAL

"Como QA, testaria: request no mesmo andar, overload, conflito de direção, fila vazia com elevador em movimento, e regressão ao inverter direção."

Frases prontas

- "I would model the domain first, then implement the movement logic."
- "The Controller orchestrates; the Elevator executes."
- "I'd use a state machine for door and movement safety rules."
- "From a QA perspective, I'd focus on edge cases around direction changes and capacity."

10. Problemas clássicos para praticar {#10-problemas-clássicos}

Faça **1 por semana** usando o mesmo framework:

#	Problema	Objetos principais	Dificuldade
---	----------	--------------------	-------------

1	Semáforo	Light, Timer, Controller	★
2	Vending Machine	Product, Inventory, CoinSlot, Controller	★★
3	Parking Lot	Spot, Ticket, Vehicle, ParkingLot	★★
4	Elevador (1 car)	Elevator, Request, Controller, Door	★★★
5	Elevador (N cars)	+ Dispatcher/Scheduler	★★★★★
6	ATM	Account, Card, Transaction, ATM	★★★★
7	Library System	Book, Member, Loan, Catalog	★★★★
8	Movie Ticket Booking	Seat, Show, Booking, Theater	★★★★

Método por problema:

1. Desenhar diagrama (10 min)
2. Listar 8 edge cases (10 min)
3. Escrever types/interfaces (15 min)
4. Implementar classes (45 min)
5. Falar solução em voz alta (5 min)

11. Exercício completo: Elevador {#11-exercício-completo-elevador}

11.1 Requisitos

- 1 elevador, andares 0–10
- Botões: hall (up/down) e cabine
- Capacidade máxima: 800 kg
- Porta só abre parado

11.2 Solução de referência

```
// types.ts
export type Direction = 'UP' | 'DOWN' | 'IDLE';
export type RequestType = 'HALL_UP' | 'HALL_DOWN' | 'CABIN';

export interface ElevatorRequest {
  floor: number;
  type: RequestType;
}

// elevator.ts
export class Elevator {
  currentFloor = 0;
  direction: Direction = 'IDLE';
  doorOpen = false;
  maxWeight = 800;
  currentWeight = 0;
  pendingStops = new Set<number>();

  addStop(floor: number): void {
    this.pendingStops.add(floor);
  }

  canMove(): boolean {
```



```

    return !this.doorOpen && this.currentWeight <= this.maxWeight;
}

openDoor(): void {
    if (this.direction !== 'IDLE') return;
    this.doorOpen = true;
}

closeDoor(): void {
    this.doorOpen = false;
}

moveOneFloor(): void {
    if (!this.canMove() || this.pendingStops.size === 0) return;

    const next = this.pickNextStop();
    if (next === null) return;

    if (next > this.currentFloor) {
        this.direction = 'UP';
        this.currentFloor++;
    } else if (next < this.currentFloor) {
        this.direction = 'DOWN';
        this.currentFloor--;
    } else {
        this.direction = 'IDLE';
        this.pendingStops.delete(this.currentFloor);
        this.openDoor();
    }
}

private pickNextStop(): number | null {
    const stops = [...this.pendingStops];
    if (!stops.length) return null;

    if (this.direction === 'UP') {
        const up = stops.filter(f => f >= this.currentFloor).sort((a, b) => a - b);
        return up[0] ?? stops.sort((a, b) => Math.abs(a - this.currentFloor) - Math.abs(b -
this.currentFloor))[0];
    }
    if (this.direction === 'DOWN') {
        const down = stops.filter(f => f <= this.currentFloor).sort((a, b) => b - a);
        return down[0] ?? stops.sort((a, b) => Math.abs(a - this.currentFloor) - Math.abs(b -
this.currentFloor))[0];
    }
    return stops.sort((a, b) => Math.abs(a - this.currentFloor) - Math.abs(b - this.currentFloor))[0];
}

// controller.ts
export class ElevatorController {
    constructor(private elevator: Elevator) {}

    handleRequest(req: ElevatorRequest): void {
        this.elevator.addStop(req.floor);
    }
}

```

```

tick(): void {
  this.elevator.moveOneFloor();
}
}

```

11.3 Cenários de teste (BDD)

Scenario: Request on same floor opens door

Given elevator is IDLE on floor 3

When CABIN request for floor 3

Then door opens without moving

Scenario: Cannot move with door open

Given door is OPEN

When tick is called

Then floor does not change

Scenario: Overweight blocks movement

Given currentWeight > maxWeight

When tick is called

Then floor does not change

Scenario: SCAN up serves higher floors first

Given elevator MOVING_UP from floor 2

And pending stops are 5, 3, 7

When moving

Then next stops are 3, 5, 7 in order

12. Plano de estudo 30 dias {#12-plano-30-dias}

Semana 1 — Fundamentos TS (Curso Maximilian)

Dia	Atividade	Tempo
1-2	Types, unions, literal types	1h/dia
3-4	Classes + interfaces	1h/dia
5	Exercício: Semáforo	2h
6	Exercício: Vending Machine	2h
7	Revisão + falar soluções em voz alta	1h

Semana 2 — Estrutura e POM (Curso Rahul Shetty + prática)

Dia	Atividade	Tempo
8-9	JS/TS fundamentals (revisão)	1h/dia
10	Page Object Model → mapear para OOP	1.5h
11-12	State machine (Refactoring Guru)	1h/dia
13	Exercício: Parking Lot	2h

14	Mock: explicar Parking Lot em 3 min	30min
----	-------------------------------------	-------

Semana 3 — Elevador e patterns

Dia	Atividade	Tempo
15-16	Implementar elevador v1	1.5h/dia
17	Adicionar Controller + fila	1.5h
18	Algoritmo SCAN + testes manuais	2h
19	Design patterns (State, Strategy, Command)	1.5h
20	Mock entrevista elevador (gravar áudio)	1h

Semana 4 — Consolidação e entrevista

Dia	Atividade	Tempo
21	ATM ou Library System	2h
22	Elevador com 2 elevadores	2h
23	10 edge cases + como testar cada um	1h
24	Framework de resposta (decorar estrutura)	1h
25-27	1 mock interview/dia (problema aleatório)	30min/dia
28-30	Revisão geral + gaps	1h/dia

Total: 35-40h em 30 dias (1-1.5h/dia)

13. Checklist pré-entrevista {#13-checklist-pré-entrevista}

24h antes

- ☐ Revisar framework de resposta (seção 9)
- ☐ Praticar 1 problema clássico em voz alta
- ☐ Preparar 3 frases "QA angle" para qualquer system design

Durante a entrevista

- ☐ Perguntar requisitos antes de responder
- ☐ Nomear objetos em voz alta
- ☐ Desenhar (ou descrever) caixas e setas
- ☐ Mencionar estados e invariantes
- ☐ Fechar com edge cases / como testaria

O que NÃO fazer

- ☐ Pular direto para `if/else` sem objetos
- ☐ Falar só de sintaxe TS sem arquitetura
- ☐ Ignorar regras de negócio (porta, capacidade)
- ☐ Esquecer seu diferencial QA

14. Recursos complementares {#14-recursos-complementares}

Recurso	URL	Uso
Refactoring Guru	refactoring.guru	Patterns + OOP visual
TypeScript Handbook	typescriptlang.org/docs	Referência oficial
XState Docs	xstate.js.org	State machines
Grokking OOP	educative.io (pago)	OOP profundo
LeetCode	leetcode.com	Lógica (nível Easy/Medium)
Exercism TypeScript	exercism.org/tracks/typescript	Exercícios guiados

Canais YouTube (buscar)

- "Elevator system design interview"
- "Parking lot low level design"
- "TypeScript OOP tutorial"

Apêndice A — Resposta ideal ao VP (elevador)

"I would start by clarifying requirements: number of floors, elevators, and capacity rules.

*Then I'd model **Elevator** (current floor, direction, door state, pending stops, weight), **ElevatorRequest** (hall vs cabin buttons), and **ElevatorController** (scheduling).*

*Movement would follow a **SCAN-style algorithm**: when going up, serve all floors above in order before reversing.*

***Safety invariants**: no movement with open doors, no movement when overweight.*

As a QA engineer, I'd test same-floor requests, direction conflicts, empty queue behavior, overload scenarios, and regression when the algorithm switches direction."

Apêndice B — Diário de estudo (template)

Data: __/__/____

Problema: _____

Tempo: __ min

Objetos identificados:

-

-

Estados:

-

Edge cases:

1.

2.

3.

O que ficou difícil:

-

Próximo passo:

-

Boa sorte, Higor. Em 30 dias de prática focada, esse tipo de pergunta passa de "travou" para "estruturou bem".

Guia gerado para uso pessoal — complemente com módulos específicos dos seus cursos Udemty conforme progresso.